

AOC 2025-2026

Introduction / rappel des concepts nécessaires
Emmanuel OSERET

Plan

- Mesure
- Optimisation
- Maîtrise environnement (Linux)
- Compilateurs
- MAQAO

Mesure et optimisation

- Optimiser une région de code nécessite :
 - de mesurer la métrique à optimiser
 - avec précision et fiabilité (évaluation stabilité)
- Ici : optimiser = réduire temps d'exécution

Principaux obstacles

- Précision du timer
 - RDTSC : précis à quelques cycles près mais biais de 20-30 cycles (amorti à partir de ~500 cycles)
 - Compteurs matériels : plusieurs secondes nécessaires
- Etat initial de la machine
 - Régime transitoire : C-states/DVFS, caches et prefetchers “froids”, pipeline non rempli
- Stabilité (variation d'une mesure à l'autre)
 - Pollution par système et autres applis

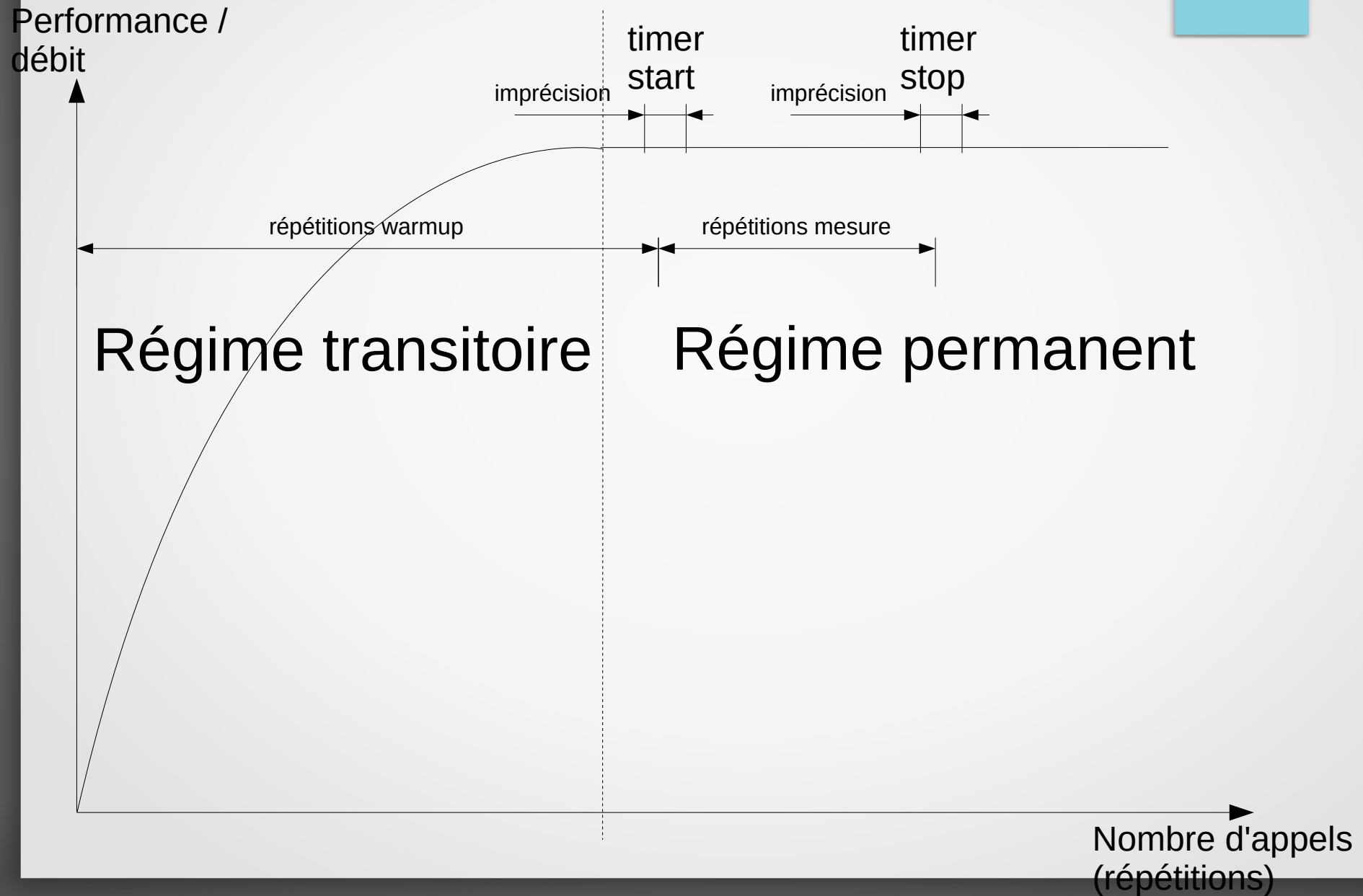
Solutions

Répétitions

- **Répétitions du noyau**
- **Avant la mesure :**
 - Pour exclure le régime transitoire
 - Appelé « warmup »
 - Surtout utile machine froide
- **Pendant la mesure :**
 - Pour amortir l'erreur du timer

Solutions aux obstacles à la précision

Répétitions



Speedup or not speedup ?

On a deux codes, lequel est optimal ?

Code 1 : 8.56 secondes

Code 2 : 8.94 secondes

Speed-up : $(8.94 - 8.56) / 8.94 = 4.2 \%$

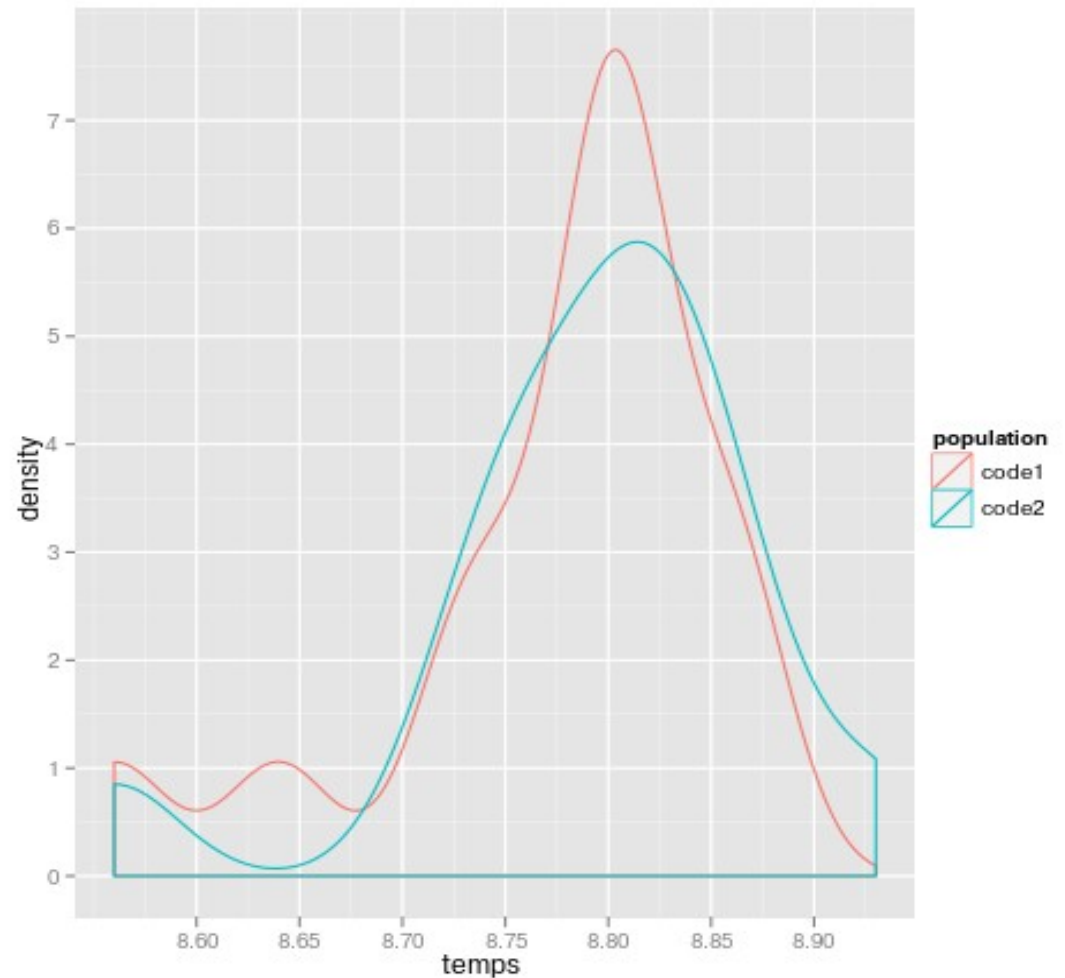
Vrai ?

Speedup or not speedup ?

Faux !

Avec 30 mesures pour chaque code, on voit que la différence n'est pas significative.

→ META-RÉPÉT.
(~ 30)



Solutions

Méta-répétitions et environnement d'exé.

- Méta-répétitions pour mesurer la stabilité
 - Répétitions du corps du driver
 - Via un script ou une boucle dans le driver
 - En toute rigueur, 31 méta-répétitions nécessaires
 - Souhaité : (médiane - minimum) / minimum < 5 %
- Environnement d'exécution le plus léger possible
 - Le plus simple et efficace : redémarrer en mode *failsafe*, *rescue*... (console uniquement, pas de réseau, mono-utilisateur)

Solutions

Solutions avancées

- Fixer processus sur 1 coeur (process pinning)
 - `taskset -c`
 - Appel POSIX
- Exécution native (et non via machine virtuelle)
- Désactiver « turbo » (voire fixer la fréquence)

Initialisation tableaux

- Politique first touch
 - Données résidentes en mem. qu'à la 1ère écriture
 - Lectures précédentes « dans le vide » (page de 0)
 - `malloc` n'initialise pas et `calloc` pointe sur page de 0
- Valeurs
 - Validation résultats
 - valeurs aléatoires mais reproductibles (**`srand (42)`**)
 - Eviter exceptions flottantes
 - nombres dénormalisés => valeurs de préférence entre 0 et 1
 - ne pas diviser par 0

Concrètement !

```
#define NB_METAS 31
```

```
int main (int argc, char *argv[])  
{  
    int m, r;
```

```
    /* Récupération arguments */  
    int size = atoi (argv[1]);  
    int repw = atoi (argv[2]);  
    int repm = atoi (argv[3]);
```

```
    for (m=0; m<NB_METAS; m++) {  
        /* Allocation */  
        float *a = malloc (...);  
        float *b = malloc (...);
```

```
        /* Init. tableaux lus */  
        init_array (size, b);
```

```
        /* Warmup */  
        if (m == 0) {  
            for (r=0; r<repw; r++)  
                kernel (size, a, b);  
        }  
        else  
            kernel (size, a, b);
```

```
        /* Répétitions */
```

```
        uint64_t t1 = rdtsc();  
        for (r=0; r<repm; r++)  
            kernel (size, a, b);  
        uint64_t t2 = rdtsc();
```

```
        /* Libération */  
        free (a);  
        free (b);
```

```
        /* Affichage performance */  
        printf ("Cycles per iter.: %.2f\  
n", (float) (t2 - t1) / (size *  
repm));  
    } // for m  
  
    return 0;  
}
```

Optimisation

- Exploitation parallélisme à tous les niveaux
- Exploitation localité spatiale et temporelle

Niveaux de parallélisme

- ILP (Instruction Level Parallelism)
 - Pipeline
 - Plusieurs instructions simult. à différents stades d'exécution
 - Unité fonctionnelle complètement pipelinée
 - Nb étages = latence (Intel : 3 cycles pour FP ADD et 5 pour FP MUL)
 - Débit = 1 instruction par cycle
 - Superscalaire
 - Intel : FP ADD || FP MUL || LD1 || LD2 || ST
- SIMD (Single Instruction Multiple Data)
- Multi-coeurs (multi-threads / multi-process)

Exploitation du parallélisme

- ILP
 - Exécution simultanée de plusieurs itérations consécutives d'une boucle
 - Optimisation typique : déroulage d'une réduction avec sommes partielles
- SIMD
 - Vectorisation (compilateur)
- Multi-coeurs
 - OpenMP (multi-threads)
 - MPI (multi-process)

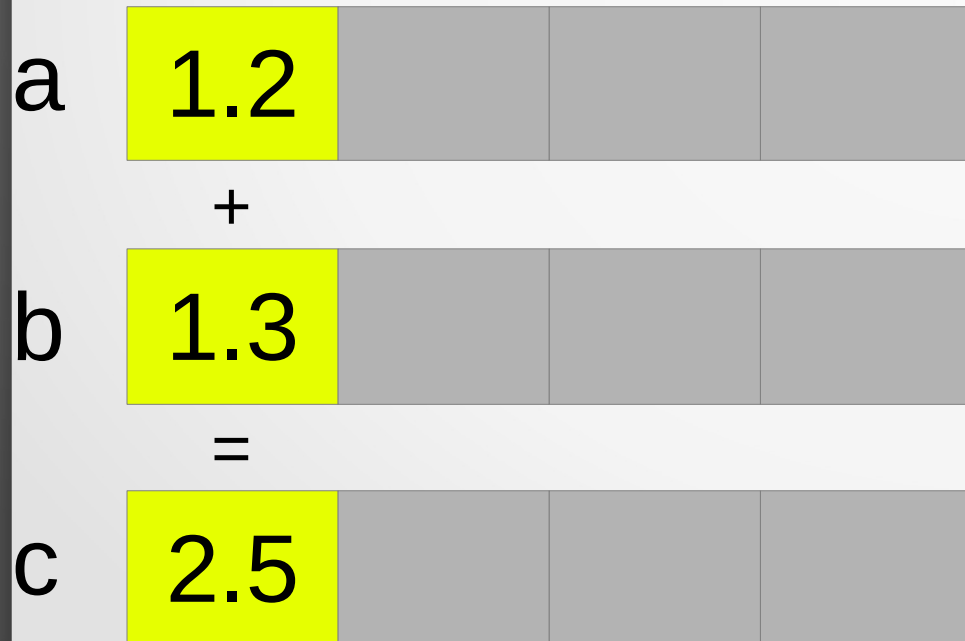
Vectorisation

- Utilisation d'instructions dites vectorielles
- Pour le même coût qu'une instruction scalaire, n éléments sont traités
 - SSE (FP/INT 128 bits) : 4 FP32
 - AVX (FP 256 bits), à partir de Sandy Bridge
 - AVX2 (INT 256 bits), à partir d'Haswell
 - AVX512 (FP/INT 512 bits), à partir de Skylake/Icelake
- Deux conditions pour boucle auto-vectorisée :
 - Vectorisable (directive IVDEP)
 - Rentable à vectoriser (directive VECTOR ALWAYS)

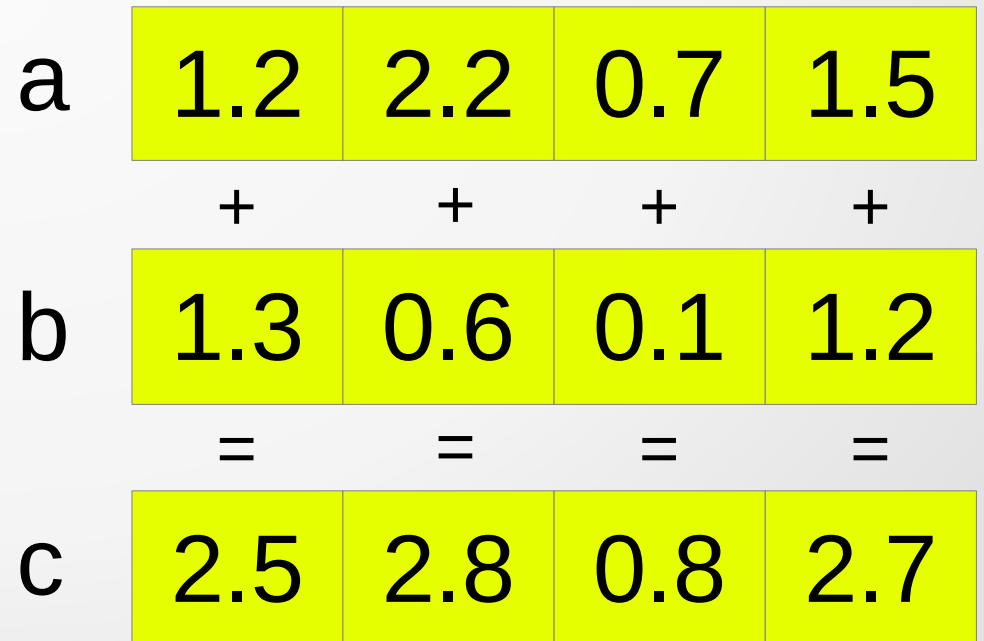
Vectorisation

```
for (i=0; i<n; i++)  
    c[i] = a[i] + b[i]; // float
```

```
VMOVSS (b), %xmm0  
VADDSS (a), %xmm0, %xmm1  
VMOVSS %xmm1, (c)
```



```
VMOVAPS (b), %xmm0  
VADDPS (a), %xmm0, %xmm1  
VMOVAPS %xmm1, (c)
```



Exploitation de la localité spatiale

- Localité spatiale : accès suivants à des adresses proches (groupement des données)
- Matériel
 - Éléments accédés par blocs (lignes de cache)
 - Chargement anticipé de ces blocs (prefetch)
- Optimisation
 - Pas unitaire (ce qui facilite aussi la vectorisation)
 - Permutation de boucles, SoA-AoS etc.

Exploitation de la localité temporelle

- Localité temporelle : accès répétés dans un futur proche (réutilisation des données)
- Matériel : caches
 - Mémoires plus petites mais plus rapides
 - Plusieurs niveaux
 - Pour instructions et données
- Optimisation
 - Fusion de boucle
 - Déroutage de boucle
 - Loop blocking

Boucle idéale

- Pas de dépendances inter-itérations (ILP)
- Pas de structures conditionnelles
- Pas unitaire (vectorisation, localité spatiale)
- Tiens dans le cache de données (localité tempo)
- Beaucoup plus de calculs que d'accès mémoire
 - Elimination bottleneck mémoire
- Ex typique : SGEMM / DGEMM

Transformation de boucles

- Transformations souvent effectuées automatiquement par le compilateur.
- Dans certains cas, le compilateur n'arrive pas à trouver la transformation.
- On ne donne que quelques exemples.
- Bibliographie : Survey of Loop Transformations Techniques, Eric LaForest 2010

Loop-invariant code motion

```
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        a[i][j] = c[i] + b[i][j] ;
```

Comment optimiser ce code ?

Loop-invariant code motion

```
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        a[i][j] = c[i] + b[i][j] ;
```



```
for (i=0; i<n; i++) {  
    K = c[i];  
    for (j=0; j<n; j++)  
        a[i][j] = K + b[i][j] ;  
}
```

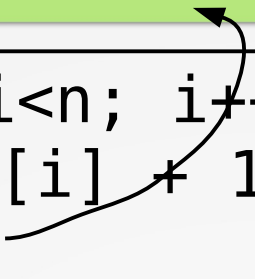
If Hoisting

```
for (i=1; i<n; i++) {  
    a[i] = a[i] + 1 ;  
    if (i<3)  
        b[i] = a[i] * c[i] ;  
    else  
        b[i] = a[i-1] * b[i-1] ;  
}
```

Comment optimiser ce code ?

If Hoisting

```
for (i=1; i<n; i++) {  
    a[i] = a[i] + 1 ;  
    if (i<3)  
        b[i] = a[i] * c[i] ;  
    else  
        b[i] = a[i-1] * b[i-1] ;  
}
```



```
for (i=1; i<3; i++) {  
    a[i] = a[i] + 1 ;  
    b[i] = a[i] * c[i] ;  
}  
for (i=3; i<n; i++) {  
    a[i] = a[i] + 1 ;  
    b[i] = a[i-1] * b[i-1] ;  
}
```

Loop interchange

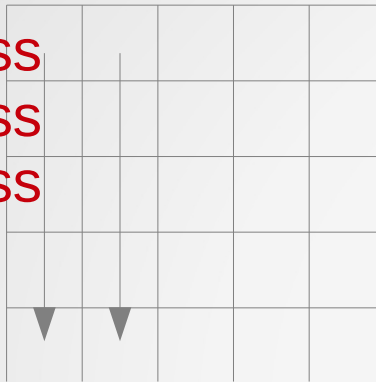
```
for (i=1; i<n; i++)  
    for (j=1; j<n; j++)  
        a[j][i] = a[j][i] * a[j][i] ;
```

Comment optimiser ce code ?

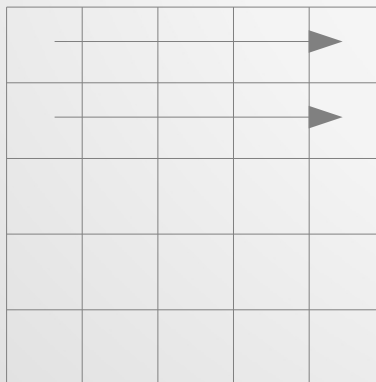
Loop interchange

Miss
Miss
Miss

...



Miss Hit Hit ...



```
for (i=1; i<n; i++)  
    for (j=1; j<n; j++)  
        a[j][i] = a[j][i] * a[j][i] ;
```



```
for (j=1; j<n; j++)  
    for (i=1; i<n; i++)  
        a[j][i] = a[j][i] * a[j][i] ;
```

Loop fusion / fission

Avantages :

Localité des accès $a[i]$

Moins de contrôle.

```
for (i=0; i<n; i++) {  
    a[i] = a[i] + c ;  
    x[i+1] = x[i]*7 + x[i+1] +  
    a[i] ;  
}
```



Avantages :

Diminue la pression sur les registres.

Peut éviter des saturations des ports.

```
for (i=0; i<n; i++)  
    a[i] = a[i] + c ;  
  
for (i=0; i<n; i++)  
    x[i+1] = x[i]*7 + x[i+1] +  
    a[i] ;
```

Comment choisir : Il faut mesurer, ça dépend de l'architecture !

Loop peeling

```
for (i=0; i<n; i++)  
    a[i] = a[i] + 2 ;  
  
for (i=2; i<n; i++)  
    a[i] = a[i] * 2 ;
```

Comment optimiser ce code ?

Loop peeling

```
for (i=0; i<n; i++)  
    a[i] = a[i] + 2 ;  
  
for (i=2; i<n; i++)  
    a[i] = a[i] * 2 ;
```

→
Peeling

```
a[0] = a[0] + 2 ;  
a[1] = a[1] + 2 ;  
for (i=2; i<n; i++)  
    a[i] = a[i] + 2 ;  
  
for (i=2; i<n; i++)  
    a[i] = a[i] * 2 ;
```

↓
Fusion

```
a[0] = a[0] + 2 ;  
a[1] = a[1] + 2 ;  
for (i=2; i<n; i++)  
    a[i] = (a[i] + 2) * 2 ;
```

Loop unrolling

```
for (i=0; i<n; i++)  
    c[i] += b[i] + a[i] ;
```

Comment optimiser ce code ?

Loop unrolling

```
for (i=0; i<n; i++)  
    c[i] += b[i] + a[i] ;
```



```
for (i=0; i<n-1; i+=2) {  
    c[i+0] += b[i+0] + a[i+0];  
    c[i+1] += b[i+1] + a[i+1];  
}  
if (n%2 == 1)  
    c[n-1] += b[n-1] + a[n-1];
```

Réduit le surcoût de contrôle.

Pour la vectorisation automatique il vaut mieux ne pas dérouler
Parfois, fait apparaître des simplifications.

Algorithmic opt. / memoization

```
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        s += exp (b[i]) + log (c[j]);
```

Comment optimiser ce code ?

Algorithmic opt. / memoization

```
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        s += exp (b[i]) + log (c[j]);
```

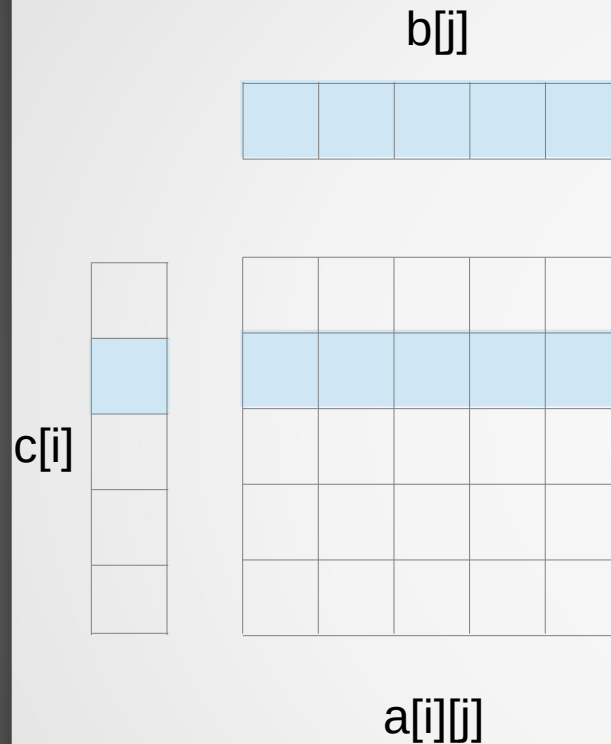


```
for (j=0; j<n; j++)  
    log_c[j] = log (c[j]);  
for (i=0; i<n; i++) {  
    const double exp_bi = exp (b[i]);  
    for (j=0; j<n; j++)  
        s += exp_bi + log_c[j];  
}
```

n appels à exp et n log au lieu de n^2

Remarque : les compilateurs devraient auto. sortir exp (b[i])

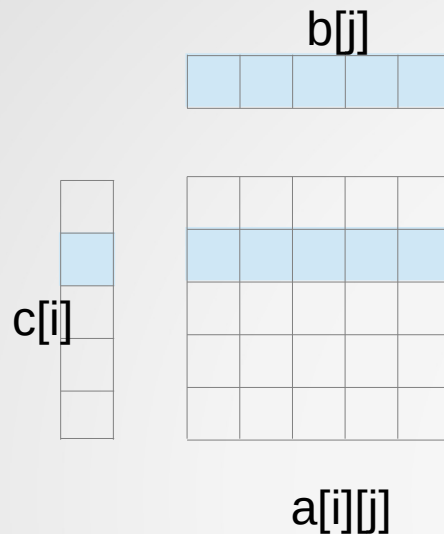
Loop tiling



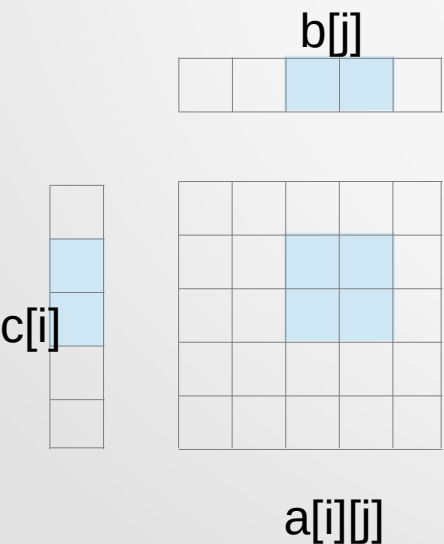
```
for (i=0; i<n; i++)  
  for (j=0; j<n; j++)  
    c[i] += a[i][j] * b[j];
```

Comment optimiser ce code lorsque n très grand ?

Loop tiling



```
for (i=0; i<n; i++)  
    for (j=0; j<n; j++)  
        c[i] += a[i][j] * b[j];
```



```
#define TILE 2  
assert(n%TILE == 0)  
for (ii=0; ii<n; ii+=TILE)  
    for (jj=0; jj<n; jj+=TILE)  
        for(i=ii; i<ii+TILE; i++)  
            for(j=jj; j<jj+TILE; j++)  
                c[i] += a[i][j] * b[j];
```

Compilateurs : zoologie

- GNU
 - gcc, g++, gfortran
 - Supporte à peu près tous les langages et processeurs existants de nos jours
 - Vectorise moins bien que les compilateurs Intel
- LLVM
 - clang, clang++, flang
 - Beaucoup plus jeune que GNU
 - flang encore relativement immature

Compilateurs : zoologie

- Intel (oneAPI)
 - Limités à C/C++/Fortran
 - Optimisés seulement pour processeurs Intel
 - Anciens : icc, icpc, ifort
 - icc et icpc ont été supprimés il y a deux ans
 - ifort est déprécié et a été supprimé
 - **Actuels : icx, icpx, ifx**
 - Basés sur LLVM

Compilateurs : zoologie

- AMD (AOCC)
 - Limités à C/C++/Fortran
 - Optimisés seulement pour processeurs AMD Zen
 - AOCC s'invoque via : clang, clang++, flang
 - Basés sur LLVM

Compilateurs : niveaux d'optimisation pour la performance

- Intel : au moins **-O2**
- GNU et LLVM : au moins **-O3**
 - Jusqu'à GNU 11, vecto. seulement à partir de -O3
- **-Ofast = -O3 + -ffast-math**
 - Nécessaire pour **vectoriser les réductions**
 - Implémentation plus rapides et vectorisées (mais potentiellement légèrement précises et non strictement conformes au standard IEEE754) des **fonctions mathématiques** sqrt, log, exp, sin/cos/tan etc. (déclarées dans math.h)

Compilateurs : spécialisation micro-arch.

- **-march=native pour cibler l'hôte de compilation**
 - Variante Intel : -xHost
- **-march=skylake pour cibler un Skylake etc.**
 - Variante Intel : -xSKYLAKE
- **Permet d'utiliser l'ensemble des instructions supportées par le processeur cible et en particulier :**
 - **Celles utilisant des vecteurs plus larges (256/512 bits)**
 - AVX2 : instructions FMA (Fused Multiply Add)
 - AVX2 : instructions GATHER
- **x86 uniquement : permet de réordonner et choisir les instructions en fonction du processeur cible (-march inclue -mtune)**

Compilateurs : autres options utiles pour la performance

- **-g pour lignes source** dans gdb et autres outils
- x86-64 : `-fno-omit-frame-pointer` pour les callchains
 - Notamment permet à des outils comme MAQAO de les récupérer à moindre coût
- **GNU : -funroll-loops** (non inclus même en O3)
- CF manpage du compilateur

Compilateurs : récap

- GNU
 - `-g -O3/-Ofast -march=native -funroll-loops`
- Intel
 - Soit `-g -O2/3 -march=native`
 - Soit `-g -O2/3 -xHost`
- LLVM/AOCC
 - `-g -O3/-Ofast -march=native`

Compilateurs : Compiler Explorer

The screenshot displays the Compiler Explorer web application in a browser. The address bar shows the URL <https://godbolt.org>. The interface includes a top navigation bar with the Compiler Explorer logo, a search bar, and links to 'Add...', 'More', and 'Templates'. A secondary bar features logos for 'Backtrace intel' and 'Solid Sands', along with 'Share', 'Policies', and 'Other' options.

The main workspace is divided into two panes. The left pane, titled 'C source #1', contains the following C code:

```
1 void daxpy (unsigned n,  
2             double *restrict a,  
3             const double *restrict b,  
4             const double *restrict c) {  
5     unsigned i;  
6     for (i=0; i<n; i++)  
7         a[i] = b[i] + c[i];  
8 }  
9
```

The right pane, titled 'x86-64 gcc 12.2 (Editor #1)', shows the compiler options '-Ofast -march=skylake' and a search bar 'Select a compiler...'. Below this is a list of 'FAVORITES' compilers:

- x86-64 gcc 12.2 (selected)
- x86-64 clang 15.0.0
- x86-64 icc 2021.7.1
- x86-64 icx 2022.2.1

Below the favorites list, the 'cc65' section is visible, showing '6502 cc65 trunk' with a star icon. The assembly output for the selected compiler is displayed on the right side of the right pane:

```
36     and     ecx, -2  
37     add     eax, ecx  
38     .L6:  
39     vmovsd  xmm0, QWORD PTR [r8+rax*8]  
40     vaddsd  xmm0, xmm0, QWORD PTR [rdx+rax*8]  
41     vmovsd  QWORD PTR [rsi+rax*8], xmm0  
42     ret
```

Compilateurs : mots clés C/C++ pour la performance

- `const` devant chaque déclaration de variable/paramètre dont la valeur ne devra pas être modifiée après init.
- `restrict` (C99) pour les pointeurs qui ne serviront jamais à référencer une variable référencée par un autre pointeur
- Particulièrement recommandés pour paramètres tableaux

```
void vadd1 (unsigned n, float a[restrict],  
           const float b[restrict]) {  
    unsigned i;  
    for (i=0; i<n; i++)  
        a[i] += b[i];  
}  
  
void vadd2 (unsigned n, float *restrict a,  
           const float *restrict b) {...}
```

Compilateurs : directives C/C++ pour la performance

- Déroulage (non portable, compilateur-spécifique)
 - gcc: `#pragma GCC unroll 4`
 - icc: `#pragma unroll(4)` et `unroll_and_jam(2)`
- Vectorisation (portable, standardisé)
 - `#pragma omp simd` : vecto. d'une boucle
 - Ajouter `reduction(+:sum)` si `sum += f(i);`
 - `#pragma omp declare simd` : vecto. d'une fonction
- Il en existe d'autres (portables ou non), notamment pour l'alignement des vecteurs. CF documentation de chaque compilateur (et OpenMP pour les directives `omp simd`)

Compilateurs : Intel oneAPI

- Téléchargement (base toolkit) :
 - <https://www.intel.com/content/www/us/en/developer/tools/oneapi/base-toolkit-download.html>
 - Utiliser la version Linux / online
 - Installation (ex si version 2024.0.1)

```
$ sudo sh ./l_BaseKit_p_2024.0.1.46.sh
```

- Configuration
 - Choisir « custom installation » (rq : pour Fortran ou MPI, besoin kit HPC)
 - Tout décocher sauf DPC++/C++ compiler et ses dépendances (CF slide suivant). Je conseille de laisser MKL coché
- Utilisation

```
$ source /opt/intel/oneapi/setvars.sh  
$ icx (C) / icpx (C++)
```

Compilateurs : Intel oneAPI

 Intel® oneAPI Base Toolkit, v. 2021.4

1 Welcome

2 Select Components

3 Integrate IDE

4 Software Improvement Program

5 Install

Develop accelerated C++ and DPC++ applications for CPUs, GPUs, and FPGAs. Toolkit includes compilers, pre-optimized libraries, and analysis tools for optimizing workloads including AI, HPC, and media.

Installation size: 11 GB Download size: 1.5 GB Expand all

☒ Intel® oneAPI Threading Building Blocks 2021.4.0 55.5 MB	☒ Intel® oneAPI DPC++/C++ Compiler 2021.4.0 4.6 GB
☒ Intel® oneAPI DPC++ Library 2021.5.0 2.7 MB	☐ Intel® Integrated Performance Primitives 2021.4.0
☐ Intel® Integrated Performance Primitives Cryptography 2021.4.0	☐ Intel® oneAPI Data Analytics Library 2021.4.0
☒ Intel® oneAPI Math Kernel Library 2021.4.0 7.7 GB	☐ Intel® oneAPI Deep Neural Network Library 2021.4.0
☐ Intel® oneAPI Collective Communications Library 2021.4.0	 Intel® FPGA Add-on for oneAPI Base Toolkit 2021.1.4.0 This component cannot be installed due to Administrator permissions are required.
☐ Intel® Distribution for Python® 2021.4.0	☐ Intel® DPC++ Compatibility Tool 2021.4.0
☐ Intel® Advisor 2021.4.0	☒ Intel® Distribution for GDB® 10.2.4 402.6 MB
☐ Intel® oneAPI Video Processing Library 2021.6.0	☐ Intel® VTune(TM) Profiler 2021.7.1

Installation Location | [Change](#)
/home/eosereet/intel/oneapi

Software Installer v.4.0.9.0-678

Compilateurs : AMD AOCC

- Téléchargement :
 - <https://www.amd.com/en/developer/aocc.html>
 - Utiliser la version correspondant à votre Linux (en général sera .deb)

- Installation (ex si version 4.2)

```
$ sudo dpkg -i aocc-compiler-4.2.0_1_amd64.deb
```

- Utilisation

```
$ source /opt/AMD/aocc-compiler-4.2.0/setenv_AOCC.sh  
$ clang (C) / clang++ (C++)
```

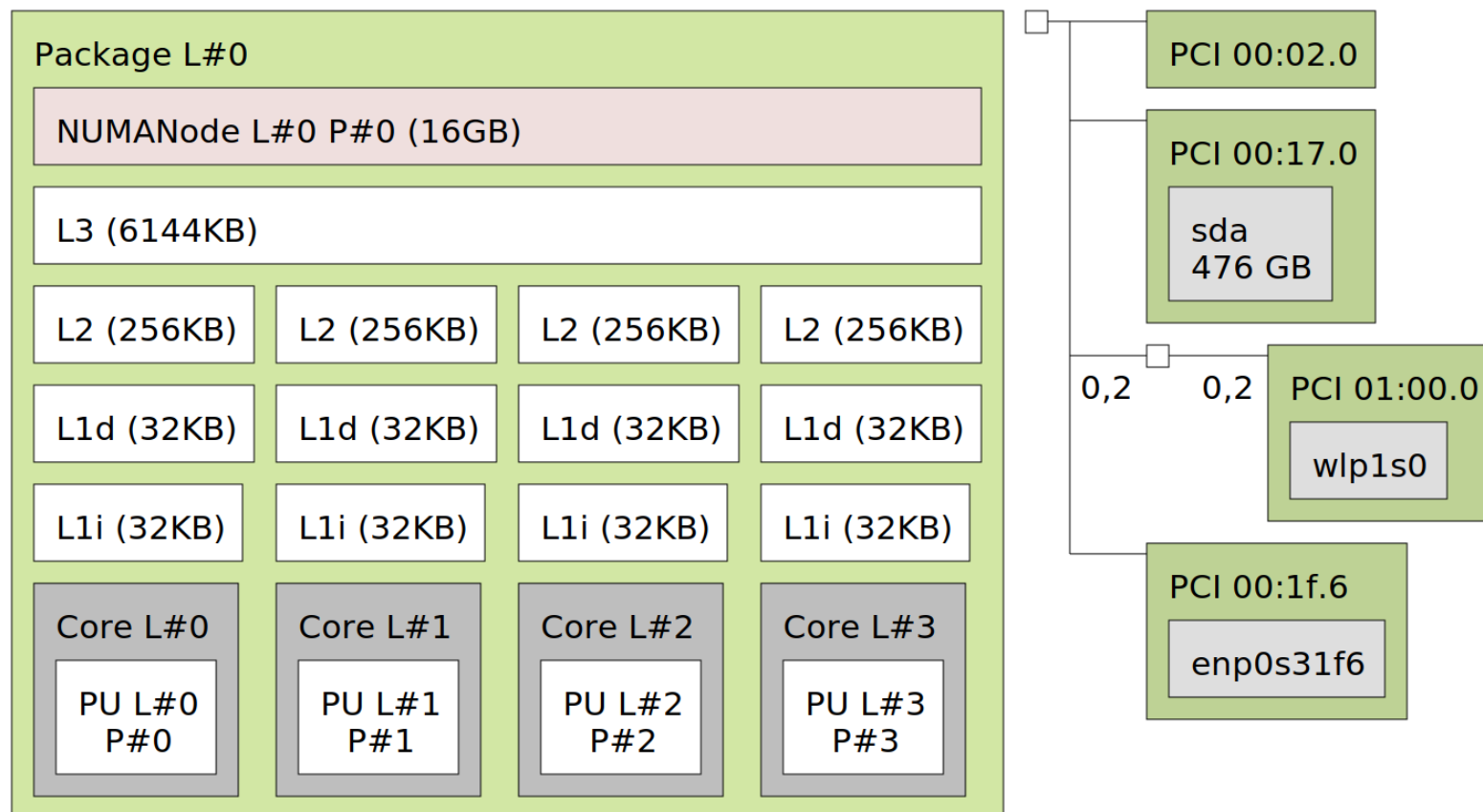
- Libm (math library) et BLAS : fourni séparément (AOCL)
 - <https://www.amd.com/en/developer/aocl.html>

Maîtrise environnement (Linux) lscpu

```
eoseret@normandy:~/MAQA0$ lscpu
Architecture : x86_64
Mode(s) opératoire(s) des processeurs : 32-bit, 64-bit
Boutisme : Little Endian
Address sizes: 39 bits physical, 48 bits virtual
Processeur(s) : 4
Liste de processeur(s) en ligne : 0-3
Thread(s) par cœur : 1
Cœur(s) par socket : 4
Socket(s) : 1
Nœud(s) NUMA : 1
Identifiant constructeur : GenuineIntel
Famille de processeur : 6
Modèle : 158
Nom de modèle : Intel(R) Core(TM) i5-7440HQ CPU @ 2.80GHz
Révision : 9
Frequency boost: enabled
Vitesse du processeur en MHz : 2800.083
Vitesse maximale du processeur en MHz : 2801,0000
Vitesse minimale du processeur en MHz : 800,0000
BogoMIPS : 5599.85
Virtualisation : VT-x
Cache L1d : 128 KiB
Cache L1i : 128 KiB
Cache L2 : 1 MiB
Cache L3 : 6 MiB
Nœud NUMA 0 de processeur(s) : 0-3
Vulnerability Itlb multihit: Mitigation: Split huge pages
Vulnerability L1tf: Mitigation; PTE Inversion; VMX conditional cache flushes, SMT disabled
Vulnerability Mds: Mitigation; Clear CPU buffers; SMT disabled
Vulnerability Meltdown: Mitigation; PTI
Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Vulnerability Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2: Mitigation; Full generic retpoline, IBPB conditional, IBRS_FW, STIBP disabled, RSB filling
Vulnerability Srbds: Mitigation; Microcode
Vulnerability Tsx async abort: Mitigation; Clear CPU buffers; SMT disabled
Drapeaux : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb invpcid_single pti ssbd ibrs ibpb stibp tpr_shadow vnmi flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm mpx rdseed adx smap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves dtherm ida arat pln pts hwp hwp_notify hwp_act_window hwp_epp md_clear flush_l1d
```

Maîtrise environnement (Linux) Istopo

Machine (16GB total)



Host: normandy

Date: jeu. 18 nov. 2021 10:45:14

Maîtrise environnement (Linux) cpupower

```
eoseret@normandy:~/MAQA0$ cpupower frequency-info
analyzing CPU 0:
  driver: acpi-cpufreq
  CPUs which run at the same hardware frequency: 0
  CPUs which need to have their frequency coordinated by software: 0
  maximum transition latency: 10.0 us
  hardware limits: 800 MHz - 2.80 GHz
  available frequency steps: 2.80 GHz, 2.80 GHz, 2.70 GHz, 2.40 GHz, 2.30 GHz, 2.20 GHz, 2.10 GHz, 1.90
GHz, 1.80 GHz, 1.70 GHz, 1.50 GHz, 1.40 GHz, 1.20 GHz, 1.10 GHz, 900 MHz, 800 MHz
  available cpufreq governors: conservative ondemand userspace powersave performance schedutil
  current policy: frequency should be within 800 MHz and 2.80 GHz.
                    The governor "powersave" may decide which speed to use
                    within this range.
  current CPU frequency: Unable to call hardware
  current CPU frequency: 900 MHz (asserted by call to kernel)
  boost state support:
    Supported: yes
    Active: yes
eoseret@normandy:~/MAQA0$ sudo cpupower frequency-set -g performance --min 2800MHz --max 2800GHz
Setting cpu: 0
Setting cpu: 1
Setting cpu: 2
Setting cpu: 3
eoseret@normandy:~/MAQA0$ cpupower frequency-info
analyzing CPU 0:
  driver: acpi-cpufreq
  CPUs which run at the same hardware frequency: 0
  CPUs which need to have their frequency coordinated by software: 0
  maximum transition latency: 10.0 us
  hardware limits: 800 MHz - 2.80 GHz
  available frequency steps: 2.80 GHz, 2.80 GHz, 2.70 GHz, 2.40 GHz, 2.30 GHz, 2.20 GHz, 2.10 GHz, 1.90
GHz, 1.80 GHz, 1.70 GHz, 1.50 GHz, 1.40 GHz, 1.20 GHz, 1.10 GHz, 900 MHz, 800 MHz
  available cpufreq governors: conservative ondemand userspace powersave performance schedutil
  current policy: frequency should be within 2.80 GHz and 2.80 GHz.
                    The governor "performance" may decide which speed to use
                    within this range.
  current CPU frequency: Unable to call hardware
  current CPU frequency: 2.80 GHz (asserted by call to kernel)
  boost state support:
    Supported: yes
    Active: yes
```

MAQAO: intro et utilisation

- Téléchargement & doc : <https://www.maqao.org>
- Installation/configuration (ex si version 2.19.0)

```
$ tar xf maqao.x86_64.2.19.0.tar.xz
$ sudo cp maqao.x86_64.2.19.0/bin/maqao /usr/local/bin
$ sudo sysctl -w kernel.perf_event_paranoid=1
$ sudo sh -c 'echo kernel.perf_event_paranoid=1 >>
/etc/sysctl.d/local.conf'
```

- Utilisation

Sans MAQAO :

```
$ ./myapp.exe 48 -i input.dat
```

Avec MAQAO (tout en ligne de commande) :

```
$ maqao OV -R1 -- ./myapp.exe 48 -i input.dat
```

Avec MAQAO (recours à un fichier de config) :

```
$ maqao OV -create-config
```

```
$ vim config.json # compléter a minima executable et run_command
```

```
$ maqao OV -R1 c=config.json
```